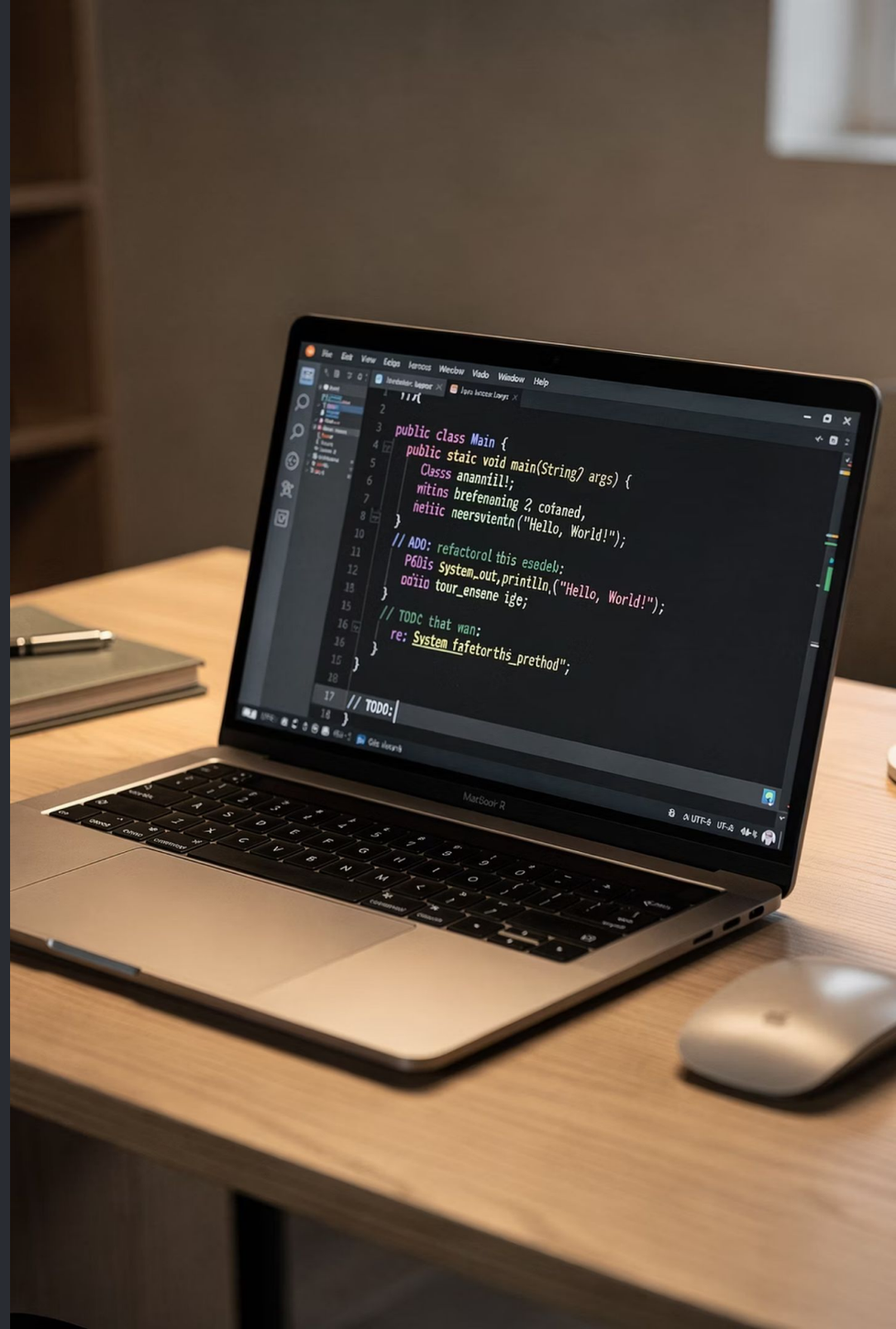
Java Fundamentals: Syntax & OOP

DAY 2

Full-Stack Development with Java, React & MongoDB — Day 2. Today you'll build a **mini console app**, write clean reusable methods, and take your first steps into Object-Oriented Programming.



Today's Agenda

This deck covers everything you need for Day 2 — from Java syntax basics to your first OOP class. Each section includes code examples, mini exercises, and common mistakes to watch out for.

01

Java Syntax Foundations

Program structure, variables, data types, operators, and Scanner input

02

Control Flow & Methods

If/else, loops, writing reusable methods with clear design rules

03

Mini App: Budget Splitter

Step-by-step console app bringing together everything you've learned

04

Intro to OOP

Classes, objects, encapsulation — your first Transaction class

Learning Outcomes

By the end of today, you'll be able to do all of the following independently:

✍️ Write Java Programs

Correct structure with class, main method, and statements

📦 Use Variables & Types

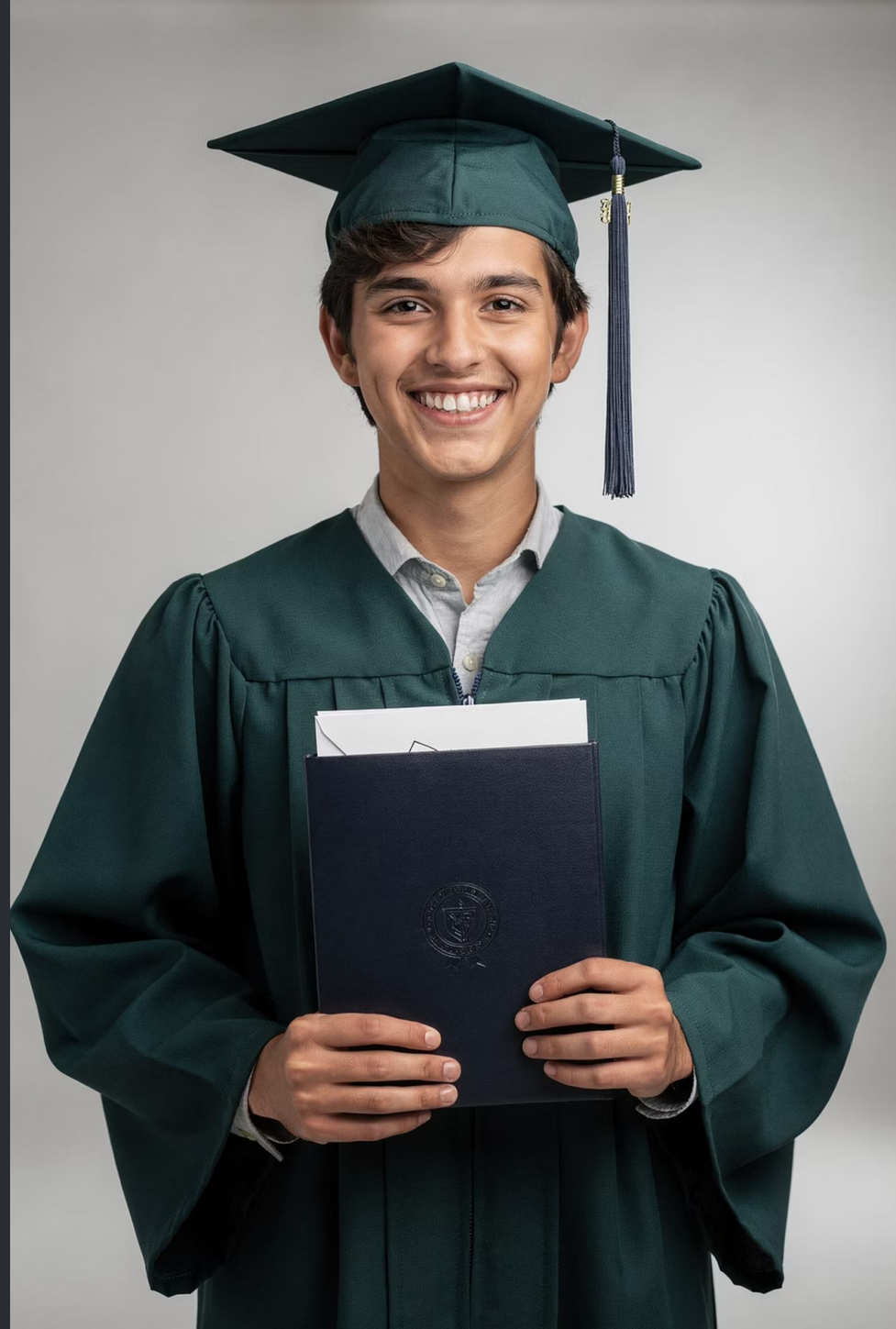
int, double, boolean, char, String — declared and used correctly

🔁 Control Flow

if/else decisions and for/while loops with real use-cases

🧩 Methods & OOP

Reusable methods, classes, objects, and simple encapsulation



Java Programme Structure

Every Java programme starts with a **class**, which is a container for your code. Inside the class, the `main` method is the entry point — the JVM always looks for it first to start execution. Each line that performs an action is a **statement**, and it ends with a semicolon.

```
public class Main {
    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }
}
```

class

Container that holds all your code. Every Java file has exactly one public class.

main method

The starting point. `public static void main(String[] args)` — memorise this signature.

statement

A single instruction, like printing text. Always ends with a `;`

Compile vs Run: The Mental Model

Java code doesn't run directly on your machine. It goes through a two-stage process that makes it both safe and portable. Understanding this pipeline helps you interpret error messages — are they **compiler errors** (syntax problems) or **runtime errors** (logic problems)?

This is why Java is called **"write once, run anywhere."** The bytecode runs on any machine that has a JVM installed — whether it's Windows, Mac, or Linux. You write the code once; the JVM handles the rest.

Variables & Data Types

A **variable** is a named container that holds a value. In Java, you must declare the **type** before the name — this tells the compiler exactly how much memory to allocate and what operations are allowed.

```
int age = 20;           // whole numbers
double price = 9.99;    // decimal numbers
boolean isActive = true; // true or false only
char grade = 'A';       // single character
String name = "Amin";   // text (note: capital S)
```

int

Whole numbers. Ages, counts, quantities. Range: −2 billion to +2 billion.

double

Decimal numbers. Prices, measurements. Use this for most numeric calculations.

boolean

Only `true` or `false`. Perfect for flags and conditions.

String

Text wrapped in double quotes. Capital S — it's an object, not a primitive.

Operators

Arithmetic

```
+   addition
-   subtraction
*   multiplication
/   division
%   remainder (modulo)
```

Comparison

```
==  equal to
!=  not equal
>   greater than
<   less than
>=  greater or equal
<=  less or equal
```

Logical

```
&&  AND (both true)
||  OR  (either true)
!   NOT (flip it)
```

🧪 Mini Example: Age Validation

```
int age = 20;
boolean isAdult = (age >= 18);
boolean isValid = (age > 0 && age < 120);

if (isAdult && isValid) {
    System.out.println("Valid adult age.");
}
```

Notice how we combine **comparison** and **logical** operators to build real validation logic. This pattern comes up constantly in back-end development.

⚠️ Don't confuse `=` (assignment) with `==` (comparison). A very common beginner mistake!

Reading Input with Scanner

The `Scanner` class lets your programme pause and wait for the user to type something. You'll use it constantly in console apps. Import it at the top of your file.

```
import java.util.Scanner;


Scanner sc = new Scanner(System.in);


System.out.print("Enter your name: ");
String name = sc.nextLine();   // reads full text line


System.out.print("Enter your age: ");
int age = sc.nextInt();        // reads integer only
```

⚠️ **Classic Gotcha — nextInt + nextLine:** After calling `sc.nextInt()`, there's a leftover newline character in the buffer. If you call `sc.nextLine()` right after, it reads that empty line instead of your input. Fix: add an extra `sc.nextLine();` after `nextInt()` to consume the leftover newline before reading text.

If / Else — Decision Making

Use `if/else` to make your programme respond differently depending on conditions. Think of it as asking a yes/no question — if the answer is yes, do this; otherwise, do that.

Basic Structure

```
if (age >= 18) {
    System.out.println("Access granted.");
} else {
    System.out.println("Too young.");
}
```

With else-if

```
if (score >= 90) {
    grade = "A";
} else if (score >= 75) {
    grade = "B";
} else {
    grade = "C";
}
```

Input Validation Pattern

A very common real-world use: check that user input is sensible before proceeding.

```
double total = sc.nextDouble();

if (total <= 0) {
    System.out.println(
      "Error: total must be > 0");
} else {
    // safe to proceed
}
```

You'll use exactly this pattern inside your **Budget Splitter** app later today.

Switch — Quick Intro for Menu Selection

When you have one variable that could match several specific values, a `switch` statement is cleaner than a long chain of `if/else if`. It's ideal for handling numbered menus.

```
int choice = sc.nextInt();

switch (choice) {
    case 1:
        System.out.println("Add expense");
        break;
    case 2:
        System.out.println("View summary");
        break;
    case 3:
        System.out.println("Exit");
        break;
    default:
        System.out.println("Invalid option. Try again.");
}
```

case

Each possible value to match against

break

Stops execution — without it, code "falls through" to the next case

default

Runs when no case matches — always include it for safety

Loops: for & while

Loops let you repeat code without copy-pasting. Choose the right loop for the situation — `for` when you know the count, `while` when you repeat until a condition changes.

for-loop — Known Count

```
// Print numbers 1 to 5
for (int i = 1; i <= 5; i++) {
    System.out.println(i);
}
```

Use when you know **how many times** to repeat. Great for processing lists or running steps a fixed number of times.

while-loop — Repeat Until Valid

```
// Keep asking until positive
int num = -1;
while (num <= 0) {
    System.out.print("Enter positive number: ");
    num = sc.nextInt();
}
System.out.println("Got: " + num);
```

Use when you want to **repeat until a condition is met** — perfect for input validation loops. You'll use this pattern heavily in the Budget Splitter.

Methods: Writing Reusable Code

A **method** is a named block of code that performs one job and can be called as many times as needed. Methods are the single most important tool for writing clean, maintainable code.

```
// Define the method
static int add(int a, int b) {
    return a + b;
}

// Call the method
int result = add(5, 3);
System.out.println(result); // prints 8
```

Reusability

Write once, call anywhere. Change the logic in one place and every caller benefits instantly.

Readability

A well-named method like `calculateSplit()` makes your code read like plain English.

Testability

Small, focused methods are easy to test in isolation — a critical skill you'll develop in Day 3.

Method Design Rules

Good methods follow a few simple rules. Violating these rules doesn't break compilation, but it creates messy, hard-to-maintain code that colleagues (and future you) will struggle with.

1 One job per method

If your method reads input AND calculates AND prints, split it into three separate methods. Single responsibility = cleaner code.

2 Meaningful names

Use verb + noun: `calculateSplit()`, `readPositiveDouble()`, `printSummary()`. Avoid vague names like `doStuff()`.

3 Return values, don't print inside

`return` the result so the caller decides what to do with it. Printing inside a method makes it impossible to reuse in a different context.

4 Validate inputs at the boundary

Check that inputs are sensible before using them. Don't assume the caller passed valid data.

Mini App: Budget Splitter

You'll build a console app that divides a total bill evenly among a group of people. Simple idea — but it touches **every concept from today**: Scanner, validation, loops, methods, and a class.

📥 Inputs

`totalAmount` — the full bill (double)

`numberOfPeople` — how many to split between (int)

📤 Output

```
Total:      RM 100.00
People:     4
Each pays:  RM 25.00
```

📋 Validation Rules

`totalAmount` must be **greater than 0**

`numberOfPeople` must be **at least 1**

If either rule is violated, show a clear error message and ask again — use a `while` loop

Budget Splitter: Step-by-Step Pseudocode

Before writing real code, always sketch the logic in plain English. This pseudocode maps directly to the Java you'll write — one step at a time.

Each step becomes either a method call or a small block of code in `main()`. When your pseudocode is clean, your Java will be clean. Spend time here before touching the keyboard.

Intro to OOP: Objects & Classes

Object-Oriented Programming is the core paradigm of Java. Before you can build anything serious — like a banking app or an e-commerce back-end — you need to understand the relationship between **classes** and **objects**.

Class = Blueprint

A class defines *what something is* — its fields (data) and methods (behaviour). It's a template. No memory is used until you create an object from it.
Like a **recipe**: it describes the ingredients and steps, but it isn't food itself.

Object = Real Instance

An object is a specific *thing created from* the class blueprint. Each object has its own copy of the fields with its own values.
Like a **cake** baked from the recipe: real, tangible, and possibly different from other cakes made from the same recipe.

Create a Class: Transaction

Let's make it concrete. A `Transaction` represents a single financial record — like a lunch purchase or a bill payment. It has **fields** to store data and a **method** to describe itself.

```
public class Transaction {

    // Fields — the data this object holds
    String description;
    double amount;

    // Constructor — called when creating an object
    Transaction(String description, double amount) {
        this.description = description;
        this.amount = amount;
    }

    // Method — what this object can do
    void printSummary() {
        System.out.println(
            "Transaction: " + description
            + " - RM" + amount
        );
    }
}
```

💡 **Tip:** The keyword `this` refers to the current object. Use `this.description` to distinguish the field from the constructor parameter of the same name.

Creating & Using Objects

Once you've defined the class, you can create as many **instances** (objects) as you need. Each one is independent and holds its own data. This is the real power of OOP — one blueprint, unlimited real-world objects.

```
// Create a Transaction object
Transaction t = new Transaction("Lunch", 12.50);

// Call its method
t.printSummary();
// Output: Transaction: Lunch - RM12.5

// Create another — completely independent
Transaction t2 = new Transaction("Taxi", 8.00);
t2.printSummary();
// Output: Transaction: Taxi - RM8.0
```

new

Keyword that allocates memory and creates the object

Transaction(...)

The constructor — sets up the object with initial values

t.printSummary()

Dot notation calls a method on a specific object instance

Encapsulation: Protecting Your Data

Encapsulation means hiding a class's internal data from outside code. You mark fields as `private` so they can't be changed directly, then provide controlled access through **getters and setters**.

Without Encapsulation ✕

```
// Anyone can set any value!
t.amount = -9999;
// No validation possible
```

With Encapsulation ✅

```
private double amount;

public void setAmount(double a) {
    if (a > 0) {
        this.amount = a;
    }
}

public double getAmount() {
    return amount;
}
```

Why It Matters

Encapsulation is the foundation of data integrity. By controlling how fields are read and written, you can:

- Prevent invalid values from corrupting your data
- Change internal implementation without breaking code that uses the class
- Add logging or validation logic in one place

For today, a light understanding is enough. You'll go deeper in Day 4 when building full back-end models.

Common Beginner Bugs

These four bugs trip up almost every new Java developer. Learn to recognise them now and save yourself hours of frustration later.

🐛 Using == for String comparison

`==` checks if two String variables point to the *same object in memory*, not if they have the same text. Always use `.equals()`.

```
// Wrong:  if (name == "Amin")
// Correct: if (name.equals("Amin"))
```

🐛 Scanner nextInt + nextLine issue

After `nextInt()`, the newline character is left in the buffer. The next `nextLine()` reads an empty string. Fix: call `sc.nextLine()` once to clear the buffer.

🐛 Integer division truncation

`5 / 2` gives `2`, not `2.5`. Both operands are int, so Java discards the decimal. Fix: use `5.0 / 2` or cast to double.

🐛 Not validating inputs

Never trust user input. A user entering `-1` for "number of people" will cause your calculation to produce nonsense. Always validate before using the value.

Today's Deliverables & Quick Quiz

📦 Submit Today

BudgetSplitter.java — with validation loops and methods

Transaction.java — with constructor and printSummary()

- Screenshot of your console output

One sentence: *"What bug confused you today and how did you solve it?"*

🧠 Quick Quiz — Check Yourself

- What is a **class**? How is it different from an **object**?
- What does a **method** do, and why should it have one job only?
- When do you use a `while` loop vs a `for` loop?

🚀 **Up Next — Day 3:** Collections (ArrayList, HashMap) + Exception Handling + Testing basics with JUnit

LAB WORKSHEET — DAY 2

Lab Setup

Before writing any code, get your workspace organised. Good folder structure is a professional habit — start it now.

01

Create your project folder

```
java/
  day2/
```

Inside your `java` workspace folder, create a sub-folder called `day2`. All today's files go here.

02

Create your main file

```
BudgetSplitter.java
```

This is your core deliverable for today. You'll also create `Transaction.java` in Lab D.

03

Compile and run a test

```
javac BudgetSplitter.java
java BudgetSplitter
```

Confirm your environment works before adding logic. Fix any setup issues now, not mid-lab.

Lab A — Warm-Up (10 minutes)

A quick hands-on warm-up to get your fingers moving and confirm your environment is working. This is low stakes — just get it running.

1

Task 1 — Print your details

Print your name and today's date to the console using `System.out.println()`. Two lines of output, neatly labelled.

```
System.out.println("Name: Amin");
System.out.println("Date: 2025-07-01");
```

2

Task 2 — Declare & print variables

Declare the following variables, assign them values, and print them in a neat format:

- `String name`
- `int age`
- `double height`

```
// Expected output:
// Name: Amin | Age: 22 | Height: 1.75
```

Lab B — Budget Splitter (Core Lab)

This is today's main challenge. Build it step by step — don't skip to the end. Each step reinforces a different concept.

1 2 3

Step 1 — Read Total

Use Scanner to read a `double`. Validate using a `while` loop: reject anything `<= 0` with a clear error message.

Step 2 — Read People

Read an `int`. Validate: must be `>= 1`. Same while-loop pattern. Remember the nextInt/nextLine gotcha.

Step 3 — Compute & Print

Calculate: `split = total / people`. Print with 2 decimal places using `String.format("%.2f", split)`.

🎯 **Expected Output:** `Total: 100.00 | People: 4 | Each pays: 25.00`

Lab C — Refactor with Methods

Your Budget Splitter works — now make it **clean**. Refactoring means restructuring your code without changing what it does. The goal: a `main()` method that reads like a simple recipe.

```
// After refactoring, main() should look like this:
public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    double total  = readPositiveDouble(sc, "Enter total bill: ");
    int    people = readMinInt(sc, "Enter number of people: ", 1);
    double split  = calculateSplit(total, people);
    System.out.printf("Each pays: RM %.2f%n", split);
}
```

Extract the following three methods:

readPositiveDouble

```
static double readPositiveDouble(
    Scanner sc, String prompt)
```

Loops until user enters a value `> 0`

readMinInt

```
static int readMinInt(
    Scanner sc, String prompt, int
min)
```

Loops until user enters a value `>= min`

calculateSplit

```
static double calculateSplit(
    double total, int people)
```

Pure calculation — no Scanner, no printing

Lab D — Add OOP: Transaction Class

Now connect your Budget Splitter to a real OOP class. Create a separate file `Transaction.java` in the same folder.

Transaction.java

```
public class Transaction {

    String description;
    double amount;

    Transaction(String description,
                double amount) {
        this.description = description;
        this.amount      = amount;
    }

    void printSummary() {
        System.out.printf(
          "Transaction: %s - RM%.2f%n",
          description, amount
        );
    }
}
```

Use it in BudgetSplitter.java

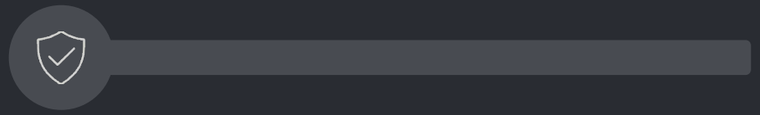
```
// After reading inputs, before printing result:
Transaction bill =
    new Transaction("Group Bill", total);

bill.printSummary();
// Output:
// Transaction: Group Bill - RM100.00
```

Then proceed to print the per-person split as before. You've now combined **procedural code** with an **OOP class** in the same programme — a major milestone!

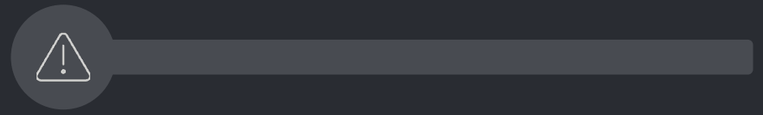
Lab E — Edge Case Challenges (If Time Permits)

If you've finished the core labs early, push your skills further with these challenges. These patterns appear constantly in production code.



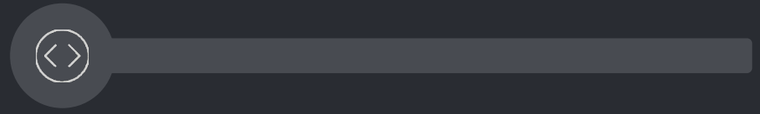
Challenge 1 — Reject Zero and Negatives

Ensure your validation loop explicitly rejects `0` and negative values for both total and people, with a descriptive error message telling the user what went wrong and what range is acceptable.



Challenge 2 — Reject Non-Numeric Input

What happens if the user types `"abc"` instead of a number? Your programme crashes with an `InputMismatchException`. Wrap your `sc.nextDouble()` in a `try/catch` block to handle this gracefully. This is a preview of Day 3's exception handling topic.



Challenge 3 — Combined Validation Method

Create a single method `readValidDouble(Scanner sc, String prompt, double min, double max)` that handles both range and type validation in one reusable utility. Then use it for all your inputs.

End-of-Day Submission Checklist

Before you pack up, make sure your submission is complete. Quality over speed — a clean, working partial solution is better than a rushed, broken full one.

1

BudgetSplitter.java

With Scanner input, validation loops, and refactored methods (`readPositiveDouble`, `readMinInt`, `calculateSplit`)

2

Transaction.java

With fields, constructor, and `printSummary()` method. Used inside BudgetSplitter before printing the split.

3

Screenshot of Output

Show your console output with valid inputs, at least one rejected invalid input, and the final formatted result.

4

Reflection Note

Write one sentence: *"What bug confused me today and how did I solve it?"* This habit builds debugging instinct over time.

🚀 **See you on Day 3:** Collections (ArrayList & HashMap), Exception Handling, and your first unit tests with JUnit.